



ENM320 Production and Operations Planning II &
BİM2006 Computer Programming IV

FINAL REPORT

GROUP-2

- 1. AHMET CAN ÇAVDAR***
- 2. ARDA SÖĞÜT***
- 3. EFE KEMAL YILMAZ***
- 4. EMRE GÖRÜCÜ***
- 5. FATİH FURKAN ŞAHİN***
- 6. İBRAHİM ALP KARINCA***
- 7. YUNUS EMRE ÖZDEMİR***

Table Of Contents

ENM320 Production and Operations Planning II & BİM2006 Computer Programming IV.....	1
<i>FINAL REPORT</i>	1
Abstract	2
Section 1: Statistical Infrastructure (InventoryStatistics.java)	3
Section 2: Main Optimization Algorithm (InventoryOptimizer.java)	4
Section 3: Solution Output	6
Section 4: How-to-Use Documentation and Software Testing	6
How-to-Use Documentation:	6
Software Testing and Verification:	6

Abstract

Effective inventory management is crucial for minimizing operational costs while maintaining high customer service levels. This report details the development of a software application designed to optimize the continuous-review (Q, R) inventory model. The primary objective of the software is to determine the optimal lot size (Q) and reorder point (R) by balancing holding, setup, and penalty costs. To achieve this, the program utilizes high-precision mathematical approximations for Z-scores and Unit Loss Function values, effectively eliminating the need for manual statistical table lookups. An iterative optimization algorithm is implemented to dynamically adjust the decision variables until convergence is reached. The accuracy and efficiency of the software were validated using a test case based on Harvey’s Specialty Shop. The results demonstrate that the software successfully computes all critical performance measures—including safety stock, average annual costs, time between orders, and service levels—providing a highly accurate and automated alternative to manual calculations.

Section 1: Statistical Infrastructure (InventoryStatistics.java)

The first part of the software development involves creating a robust statistical infrastructure. Since the (Q,R) inventory model requires continuous lookups for Z-scores and Loss Function values, we developed a dedicated utility class named “InventoryStatistics.java”. This class eliminates the need for manual table lookups by implementing high-precision mathematical approximations.

```
public static double lookupZTable(double p) {
    if (p <= 0 || p >= 1) return (p <= 0) ? -8.0 : 8.0;

    double a1 = -3.969683028665376e+01, a2 = 2.209460984245205e+02, a3 = -2.759285104469687e+02;
    double a4 = 1.383577518672690e+02, a5 = -3.066479806614716e+01, a6 = 2.506628277459239e+00;
    double b1 = -5.447609879822406e+01, b2 = 1.615858368580409e+02, b3 = -1.556989798598866e+02;
    double b4 = 6.680131188771972e+01, b5 = -1.328068155288572e+01;
    double c1 = -7.784894002430293e-03, c2 = -3.223964580411365e-01, c3 = -2.400758277161838e+00;
    double c4 = -2.549732539343734e+00, c5 = 4.374664141464968e+00, c6 = 2.938163982698783e+00;
    double d1 = 7.784695709041462e-03, d2 = 3.224671290700398e-01, d3 = 2.445134137142996e+00, d4 = 3.754408661907416e+00

    double q, r, z;
    if (p < 0.02425) {
        q = Math.sqrt(-2 * Math.log(p));
        z = (((((c1 * q + c2) * q + c3) * q + c4) * q + c5) * q + c6) /
            (((d1 * q + d2) * q + d3) * q + d4) * q + 1);
    } else if (p <= 1 - 0.02425) {
        q = p - 0.5;
        r = q * q;
        z = (((((a1 * r + a2) * r + a3) * r + a4) * r + a5) * r + a6) * q /
            (((b1 * r + b2) * r + b3) * r + b4) * r + b5) * r + 1);
    } else {
        q = Math.sqrt(-2 * Math.log(1 - p));
        z = -((((c1 * q + c2) * q + c3) * q + c4) * q + c5) * q + c6 /
            (((d1 * q + d2) * q + d3) * q + d4) * q + 1);
    }
    return z;
}
```

Figure 1: We present the lookup Z-Table method. This function implements Acklam’s approximation to calculate the inverse standard normal cumulative distribution. It allows the program to dynamically determine the Z-score for any given service level (Probability of No Stockout) with a precision of 10^{-9} .

```
public static double lookupLossFunction(double z) {
    double pdf = Math.exp(-0.5 * z * z) / Math.sqrt(2 * Math.PI);
    double cdf = 0.5 * (1 + errorFunction(z / Math.sqrt(2)));
    return pdf - z * (1 - cdf);
}

/**
 * Approximation of the error function (erf) used for CDF calculations.
 */
private static double errorFunction(double x) {
    double sign = (x < 0) ? -1 : 1;
    x = Math.abs(x);
    double t = 1.0 / (1.0 + 0.3275911 * x);
    double a1 = 0.254829592, a2 = -0.284496736, a3 = 1.421413741, a4 = -1.453152027, a5 = 1.061405145;
    double y = 1.0 - (((((a5 * t + a4) * t) + a3) * t + a2) * t + a1) * t * Math.exp(-x * x);
    return sign * y;
}
```

Figure 2: Illustrates the implementation of the Unit Loss Function $L(z)$. This method is critical for calculating the expected shortage per cycle ($n(R)$). It integrates the probability density function (PDF) and the cumulative distribution function (CDF) to provide accurate values for the iterative optimization process.

Section 2: Main Optimization Algorithm (InventoryOptimizer.java)

The core of the project is implemented in the InventoryOptimizer.java class. This program is responsible for handling user inputs, executing the iterative algorithm to find the optimal decision variables, and calculating the final performance measures.

```
Run | Debug | Run main | Debug main
public static void main(String[] args) {
    try (Scanner scanner = new Scanner(System.in)) {
        /**
         * 1. INPUT PHASE
         * Requesting cost and demand parameters from the user.
         */
        System.out.print(s: "Enter unit cost ($): ");
        double unitCost = scanner.nextDouble();

        System.out.print(s: "Enter ordering cost (K): ");
        double orderingCost = scanner.nextDouble();

        System.out.print(s: "Enter penalty cost (p): ");
        double penaltyCost = scanner.nextDouble();

        System.out.print(s: "Enter annual interest rate (e.g., 0.25): ");
        double interestRate = scanner.nextDouble();

        System.out.print(s: "Enter lead time (months): ");
        double leadTime = scanner.nextDouble();

        System.out.print(s: "Enter average demand during lead time (mu): ");
        double meanDemand = scanner.nextDouble();

        System.out.print(s: "Enter standard deviation of lead time demand (sigma): ");
        double stdDev = scanner.nextDouble();

        /**
         * 2. PRE-CALCULATIONS
         * Calculating annual demand and holding cost.
         */
        double annualDemand = meanDemand * (12.0 / leadTime);
        double holdingCost = interestRate * unitCost;

        /**
         * 3. INITIALIZATION
         * Setting up initial values for the iteration loop.
         */
        double expectedShortage = 0.0;
        double previousZ = -1.0;
        double currentZ = 0.0;

        int iterations = 0;
        int maxIterations = 1000; // Failsafe limit to prevent infinite loops

        double lotSize = 0.0;
        double serviceLevel = 0.0;
    }
}
```

Figure 3: We illustrate the input phase of the software. The program successfully requests system parameters such as unit cost, ordering cost, penalty cost, and lead time metrics from the user. Subsequently, it performs the required pre-calculations to determine the annual demand and holding cost before initializing the algorithm loop.

```

while (Math.abs(previousZ - currentZ) > 0.0001 || iterations == 0) {

    // Failsafe mechanism
    if (iterations >= maxIterations) {
        System.out.println("\nWARNING: Maximum iteration limit (" + maxIterations + ") reached.");
        System.out.println(x: "The results might not be fully converged.");
        break;
    }

    previousZ = currentZ;
    iterations++;

    // Calculate Order Quantity (Q)
    lotSize = Math.sqrt((2.0 * annualDemand * (orderingCost + (penaltyCost * expectedShortage))) / holdingCost);

    // Calculate Service Level Probabilities
    double stockoutProb = (holdingCost * lotSize) / (penaltyCost * annualDemand);
    serviceLevel = 1.0 - stockoutProb;

    // Lookup statistical values from the separate utility class
    currentZ = InventoryStatistics.lookupZTable(serviceLevel);
    double lossValue = InventoryStatistics.lookupLossFunction(currentZ);

    // Update n(R)
    expectedShortage = stdDev * lossValue;
}

```

Figure 4: Demonstrates the core iterative loop of the inventory optimization model. The algorithm continuously calculates the order quantity (Q) and updates the expected shortage by fetching statistical values from the utility class. To ensure absolute precision and prevent infinite loops, the iteration continues until the Z-score stabilizes with a tolerance of 0.0001, bounded by a maximum iteration failsafe.

```

long finalLotSize = Math.round(lotSize);
long reorderPoint = Math.round(meanDemand + (currentZ * stdDev));
long safetyStock = reorderPoint - Math.round(meanDemand);

double setupCost = orderingCost * (annualDemand / finalLotSize);
double annualHoldingCost = holdingCost * ((finalLotSize / 2.0) + safetyStock);
double annualPenaltyCost = penaltyCost * (annualDemand / finalLotSize) * expectedShortage;

double timeBetweenOrders = (finalLotSize / annualDemand) * 12.0;
double unmetDemandProportion = expectedShortage / finalLotSize;

/**
 * 6. OUTPUT RESULTS
 * Displaying all calculated performance measures.
 */
System.out.println(x: "\n--- FINAL OPTIMIZATION RESULTS ---");
System.out.println("Optimal Lot Size (Q): " + finalLotSize);
System.out.println("Reorder Point (R): " + reorderPoint);
System.out.println("Iterations Performed: " + iterations);
System.out.println("Safety Stock: " + safetyStock);
System.out.printf(format: "Annual Holding Cost: $%.2f\n", annualHoldingCost);
System.out.printf(format: "Annual Setup Cost: $%.2f\n", setupCost);
System.out.printf(format: "Annual Penalty Cost: $%.2f\n", annualPenaltyCost);
System.out.printf(format: "Time Between Orders (Months): %.3f\n", timeBetweenOrders);
System.out.printf(format: "Probability of No Stockout: %.4f\n", serviceLevel);
System.out.printf(format: "Proportion of Demands Not Met: %.4f\n", unmetDemandProportion);

```

Figure 5: Once the optimal lot size and reorder point are obtained, the software computes all requested performance measures, including the safety stock, average annual costs, and service level proportions. All variables are then formatted and displayed to the user on a single screen as required.

Section 3: Solution Output

```
Enter unit cost ($): 20
Enter ordering cost (K): 100
Enter penalty cost (p): 20
Enter annual interest rate (e.g., 0.25): 0.25
Enter lead time (months): 4
Enter average demand during lead time (mu): 500
Enter standard deviation of lead time demand (sigma): 100

--- FINAL OPTIMIZATION RESULTS ---
Optimal Lot Size (Q): 290
Reorder Point (R): 666
Iterations Performed: 6
Safety Stock: 166
Annual Holding Cost: $1555.00
Annual Setup Cost: $517.24
Annual Penalty Cost: $207.81
Time Between Orders (Months): 2.320
Probability of No Stockout: 0.9517
Proportion of Demands Not Met: 0.0069
```

Figure 6: To verify the accuracy of the algorithm, we tested the software using the data from Harvey's Specialty Shop. Figure 6 shows the execution output. The software successfully completed the optimization and presented the optimal lot size, reorder point, and all corresponding performance measures on a single screen. Also, it is more precise than on hand solution because we used more accurate rounding methods and z-values.

Section 4: How-to-Use Documentation and Software Testing

How-to-Use Documentation: The software is designed to be highly user-friendly via a sequential command-line interface. To use the program, the user simply runs the `InventoryOptimizer` class. The system will prompt the user to input the following parameters step-by-step: unit cost, ordering cost, penalty cost, annual interest rate, lead time (in months), average demand during lead time, and the standard deviation of lead time demand. After the final input is entered, the software automatically performs the pre-calculations, runs the iterative algorithm, and outputs a formatted summary screen detailing the optimal Lot Size (Q), Reorder Point (R), Safety Stock, and all relevant annual costs.

Software Testing and Verification: To verify the correctness of the code and the implemented algorithm, the software was rigorously tested using the established data from Harvey's Specialty Shop. The execution output confirmed that the software successfully completed the optimization process and presented the optimal lot size, reorder point, etc. Furthermore, the software provides a higher degree of precision compared to traditional manual hand solutions because it utilizes highly accurate, continuous mathematical approximations for Z-values and exact programmatic rounding methods.